Introductory Programming and Problem-Solving Skills

Week -1

Basanta Singh

Weekly Study Hours

Navigation

Weekly Study Hours

Credits Hours Breakdown

- Each of your module carries 20 Credits
- Total Learning Time: 20 Credits x 10 hrs= 200 hrs. (Since 1 Credit = 10 Hours of Learning Hours)
- **Guided Learning (Comprises of LTW sessions)**:
  35% of Total Learning Hours (200 hrs.) = 70 Hours
- **Unguided Learning (Comprises of self studies and time taken for the assessments):**
  65% of Total Learning Hours (200 hrs.) = 130 Hours

Module Structure

Navigation

Module Structure

01

Lecture (2 hrs.)

02

Tutorial (2 hrs.)

03

Workshop (2.5 hrs.)

6.5 hours of total class time per module per week
6.5 hours x 12 Weeks = 78 hrs per module per semester

Some Practice

Navigation

Some Practice

Some of the practices which we will be doing throughout the semester

- Lab Report
- Case Studies
- Pair Programming
- Individual Session Every Week
- Power Test

So what's it all about?

Navigation

Learning Outcomes

At the end of the module you should be able to:

LO1 - Understand the fundamental concepts common to modern programming languages.

LO2 - Interpret and understand small programs

LO3 - Develop algorithms to programmatically solve problems.

LO4 - Implement algorithms using the basic features of a modern programming language.

How will you be assessed?

Navigation

How will you be assessed ?

Component 1 (Coursework):

80% - Coding Challenges on Canvas

10% - Presentation/Viva

Component 2 (In-class Test):

10% - Written Examination (Assessment week, date will be given later) in-class test.

Assessment at undergraduate level require a minimum of 40% to pass.

Note: You must pass both components to pass the module!!!

What’s expected of you?

- Be **punctual** for lectures, tutorials and workshops.
- **Switch off** your electronic devices or put them on silent.
- Make sure you **attend** all the sessions. Please send one of us an email if you are unable to make it.
- Bring a **pen and paper** or smart device to sessions so that you can take notes.
- You should spend **at least an additional 5 hours** a week studying in your own time.
- Complete all of the **workshop exercises and coursework tasks**.

Module Schedule

Navigation

Module Schedule

Week Commencing	Topic
Week 1	Problem Solving Skills
Week 2	Introduction To Python
Week 3	Data & Expressions
Week 4	Making Decisions
Week 5	Iteration
Week 6	Data Structures
Week 7	Function

Week Commencing	Topic
Week 8	File Handling and Exception Handling
Week 9	Modular Design
Week 10	Recursion
Week 11	Object Oriented Programming
Week 12	Exam Week
Week 13	Assessment Week
Week 14	Assessment Week

Note: Subject To Change

1. Computer System

Navigation

1. Computer System

- A computer system is a basic, full-featured, and operational hardware and software configuration that includes all necessary components to carry out computing operations.
- **Computer Hardware:** The term "computer hardware" refers to all of the tangible parts of a digital or analogue computer.
- **Computer Software:** Basically sequence of machine instructions. Not perceivable by touch. MS office package, Photoshop, games, windows, Ubuntu and so on.

This course is all about software development

1.1 Program Translation

Navigation

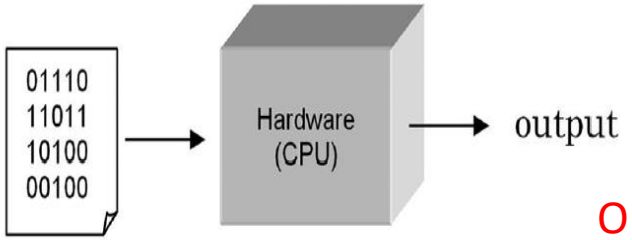
1. Computer System

1.1. Program Translation

There are two types programming languages

- **Low level languages:** instructions written using 0s and 1s.
- **High level languages:** instructions written using characters and words. E.g. x = 2

A central processing unit (CPU) is designed to interpret and execute a specific set of instructions represented in binary form (i.e., 1s and 0s) called **machine code**.



Only programs in machine code can be executed by a CPU.

1.2 Program Translators

Navigation

1. Computer System

1.1. Program Translation

1.2. Program Translators

Writing programs at this "**low level**" is tedious and error-prone. Therefore, most programs are written in a "high-level" programming language such as Python. Since the instructions of such programs are not in machine code that a CPU can execute, a translator program must be used.

There are two fundamental types of translators:

- **Compiler:** Translates programs into machine/bytecode to be executed by the CPU
- **Interpreter:** Executes instructions in place of the CPU

1.2.1 Compilers

Navigation

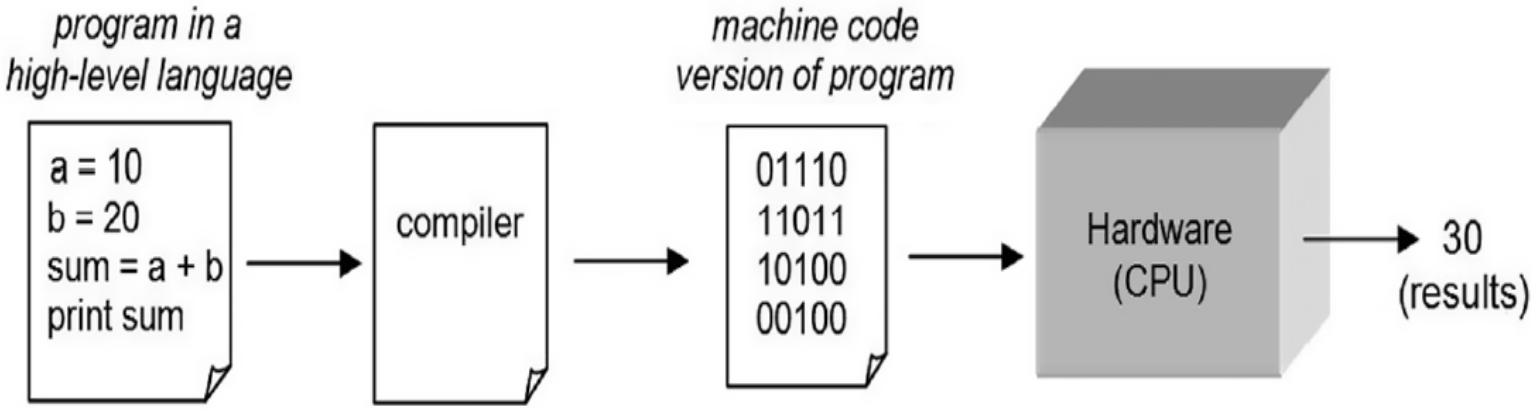
1. Computer System

1.1. Program Translation

1.2. Program Translators

1.2.1 Compilers

- Compiled programs generally **execute faster** than interpreted programs because the source code only needs to be **compiled once**.



1.2.2 Interpreters

Navigation

1. Computer System

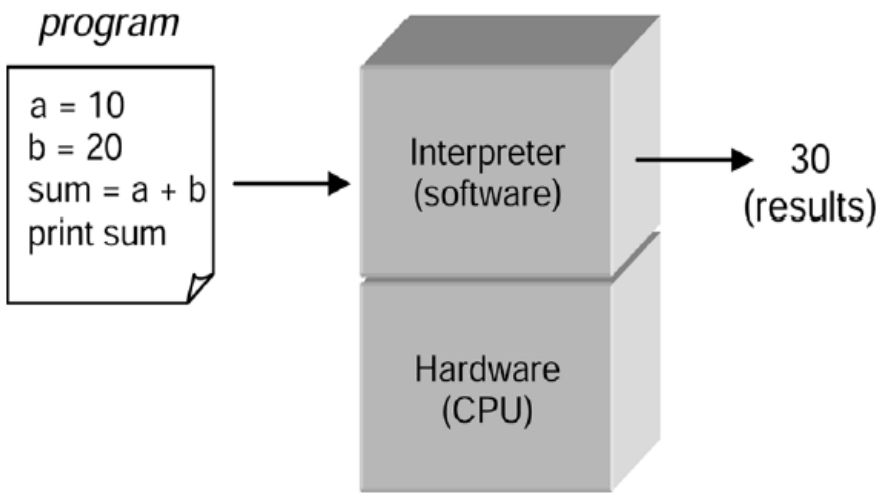
1.1. Program Translation

1.2. Program Translators

1.2.1 Compilers

1.2.2 Interpreters

- **An interpreter can immediately execute instructions as they are entered**. This is referred to as **interactive mode**. This is a very useful feature for program development. Python do use CPython interpreters for the translation.



2. Why Computer Science?

Navigation

1. Computer System
   - 1.1. Program Translation
   - 1.2. Program Translators
   - 1.2.1 Compilers
   - 1.2.2 Interpreters
   - 2. Why Computer Science ?

- Computer Science is fundamentally about **computational problem solving**.
- Programming and computers are only tools in the field of computing. The field has tremendous breadth and diversity. Areas of study include:

- Systems Programming
- Computer Architecture
- Human Computer interaction (HCI)
- Robotics
- Artificial Intelligence
- Software Engineering
- Database Management
- Data Mining
- Computer Networks
- Graphics
- Computer Simulations
- Cyber security
- Programming Language Design
- Virtual/Augmented Reality

2.1 Computational Problem Solving

Navigation

1. Computer System
   1.1. Program Translation
   1.2. Program Translators
   1.2.1 Compilers
   1.2.2 Interpreters
2. Why Computer Science ?
2.1 Computational Problem Solving

- Two things are needed to solve computational problems:
  - A **representation** that captures the relevant aspects of the problem.
  - An **algorithm** that solves the problem based on this **representation**
- This allows us to take complex problems, understand them and develop solutions. We can present these solutions in a way that a computer or human can understand.

2.2 Abstraction

Navigation

2. Why Computer Science ?

2.1 Computational Problem Solving

2.2 Abstraction

- An abstraction is a representation that leaves out details that aren't needed in order to focus on those that are.
- You see abstraction all the time. For example, maps are abstract representations.
- They include limited detail and highlight the important parts such as **roads** and **railways**.

2.3 Which is Easier To Read?

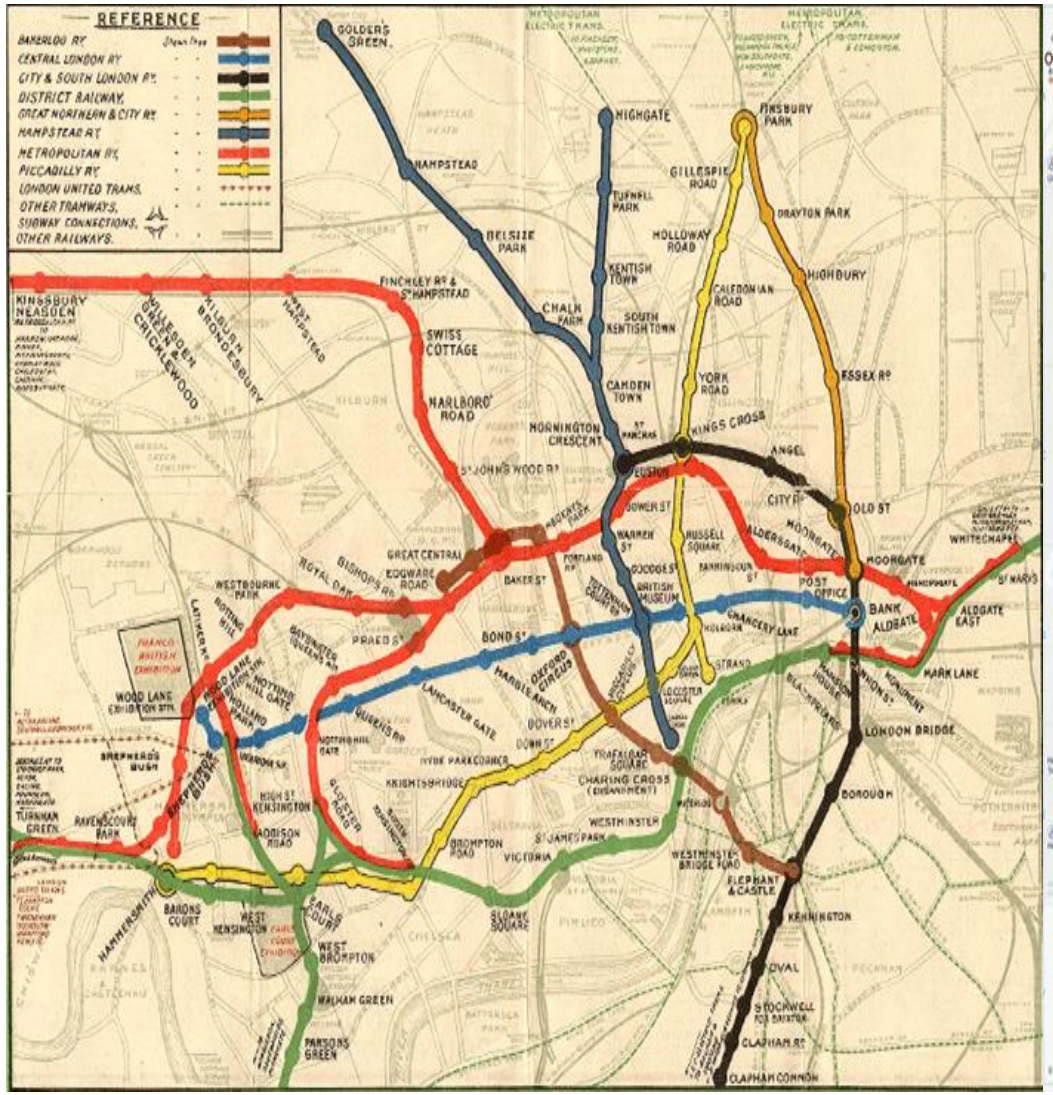
Navigation

2. Why Computer Science ?

2.1 Computational Problem Solving

2.2 Abstraction

2.3 Which is Easier To Read ?



3.1 Man, Cabbage, Goat, Wolf Problem

- A man lives on the east side of a river. He wishes to bring a cabbage, a goat and a wolf to a village on the west side of the river to sell.
- However, his boat is only big enough to hold himself and either the cabbage goat or wolf. In addition the man cannot leave the goat alone with the cabbage because the goat will eat it and he cannot leave the wolf alone with the goat because it would eat the goat.
- How does the man solve this problem?

3.1 Man, Cabbage, Goat, Wolf Problem

- There is a simple algorithmic approach for solving this problem by simply trying all of the possible combinations of items that could be rowed back and forth.

This is often known as a **brute force approach**.

- **How could we represent this problem?** Remember, whatever representation we use should only include the aspects of the problem that are necessary to find a solution.

- For example, do we need to know the colour of the boat or the name of the man?

- In this case, the only information relevant to the problem is the location of each item. Therefore we can abstract away these other details.

3.1 Man, Cabbage, Goat, Wolf Problem

- The actual problem is to determine how the man can row objects across the river with certain constraints on which pairs of objects can be left alone.

- The computational problem then is to find a way to convert our starting representation with the constraint that there are certain invalid states that result in failure.

Man	Cabbage	Goat	Wolf
{E,	E,	E,	E}

→

Man	Cabbage	Goat	Wolf
{W,	W,	W,	W}

3.1 Man, Cabbage, Goat, Wolf Problem

Navigation

3. Algorithm and Problem Solving

3.1 Man, Cabbage, Goat, Wolf Problem

- From the first state there are three possible moves, only one of which results in valid state:

Man {E, Cabbage E, Goat E, Wolf E}

Man {W, Cabbage W, Goat E, Wolf E}
Man rows cabbage across

INVALID
Goat left alone with wolf

Man {W, Cabbage E, Goat W, Wolf E}
Man rows goat across

VALID
Cabbage left alone with wolf

Man {W, Cabbage E, Goat E, Wolf W}
Man rows wolf across

INVALID
Cabbage left alone with goat

3.2 Algorithm

Navigation

3. Algorithm and Problem Solving

3.1 Man, Cabbage, Goat, Wolf Problem

3.2 Algorithm

- An **algorithm** is a finite number of clearly described steps that can be followed to produce a desired result for given input in a finite amount of time (that is, it eventually terminates).

- **Computer algorithms are central to computer science.** They provide step-by-step methods of computation that a machine can carry out.

- **Computers can execute a large number of instructions very quickly and reliably without error**, making algorithms and computers are a perfect match! However, **the computation that a given computer performs is only as good as the underlying algorithm used.**

3.2.1 Winter Tea

Navigation

3. Algorithm and Problem Solving

3.1 Man, Cabbage, Goat, Wolf Problem

3.2 Algorithm

3.2.1 Example

Step 1: Put the teabag in a cup.

Step 2: Fill the kettle with water.

Step 3: Boil the water in the kettle.

Step 4: Pour some of the boiled water into the cup.

Step 5: Add milk to the cup.

Step 6: Add sugar to the cup.

Step 7: Stir the tea.

Step 8 :Drink the tea.

3.3 Flowchart

Navigation

3. Algorithm and Problem Solving

3.1 Man, Cabbage, Goat, Wolf Problem

3.2 Algorithm

3.2.1 Example

3.3 Flowchart

(Dictionary) : A schematic representation of a sequence of operations, as in a manufacturing process or computer program.

(Technical) A graphical representation of the sequence of operations in an information system or program.

- Information system flowcharts show how data flows from source documents through the computer to final distribution to users.
- Program flowcharts show the sequence of instructions in a single program or subroutine. Different symbols are used to draw each type of flowchart.

3.3.1 Let’s Understand Through Example

Navigation

3. Algorithm and Problem Solving

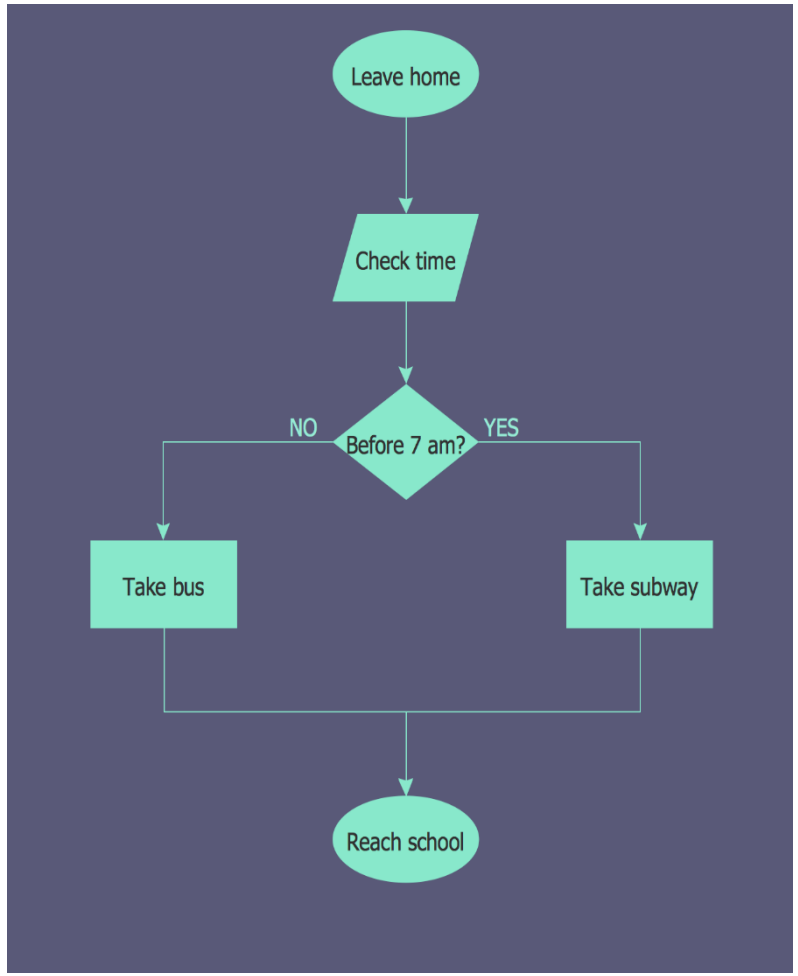
3.1 Man, Cabbage, Goat, Wolf Problem

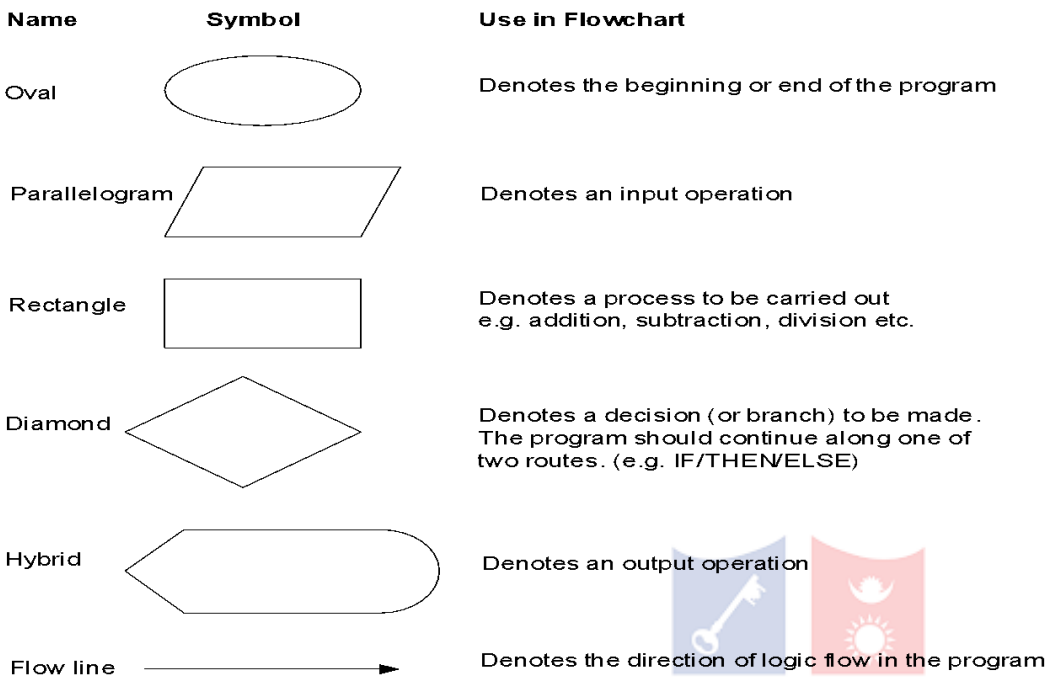
3.2 Algorithm

3.2.1 Example

3.3 Flowchart

3.3.1 Example





4. Introduction To Python Programming

4.1 Python Programming

Navigation

4. Introduction To Python Programming

4.1 Python Programming

- The Python Programming Language was created by Guido van Rossum. It was first released in the early 1990s.
- Its name comes from a 1970s British comedy sketch show called Monty Python's Flying Circus.
- Companies and organizations that use Python include YouTube, Google, Yahoo and NASA.

4.2 How Popular?

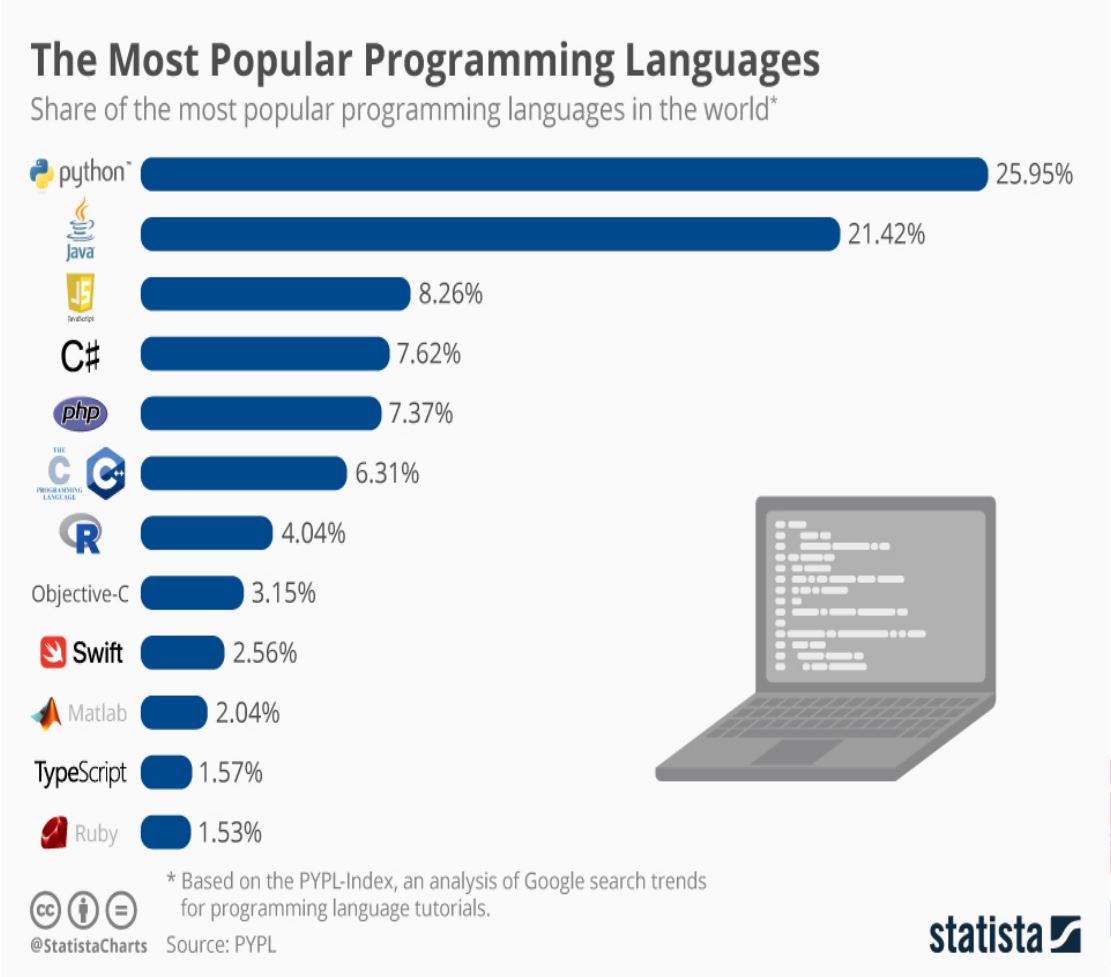
Navigation

4. Introduction To Python Programming

4.1 Python Programming

4.2 How Popular ?

As per 2019 data, there are over 9 million Python developers



4.3.1 Python Feature

Navigation

4. Introduction To Python Programming

4.1 Python Programming

4.2 How Popular ?

4.2.1 Why IDLE Editor ?

- **Simple Syntax**

Python programs are clear and easy to read

- **Interpreted Language**

Python instructions can be executed interactively

- **Powerful Programming Features**

Can accomplish significant computation with few instructions

- **Numerous Python Modules Provide Additional Capabilities**

Capabilities that can be incorporated into a Python program

4.4 Programming Paradigm

Navigation

4. Introduction To Python Programming

4.1 Python Programming

4.2 How Popular ?

4.2.1 Why IDLE Editor ?

4.2.2 Programming Paradigm

Programming languages fall into a number of programming **paradigms**.

The two major programming paradigm in use today are:

- Procedural (imperative) programming
- Object Oriented Programming

- Each provides a different way of thinking about computation. While most of the programming language supports one paradigm, Python supports both procedural and object oriented programming.

Note: We will start with the procedural aspects of python

4.3 The IDLE Editor

Navigation

4. Introduction To Python Programming

4.1 Python Programming

4.2 How Popular ?

4.3 The IDLE Editor

- IDLE is an integrated development environment (IDE). An IDE is a bundled set of software tools for program development. This typically includes:

Editor : For creating and modifying programs

Translator : For executing programs

Debugger : For taking control of the execution of a program to aid in finding program errors

Note: You will use IDLE to develop programs on this module.

4.3.1 The Python Shell

Navigation

4. Introduction To Python Programming

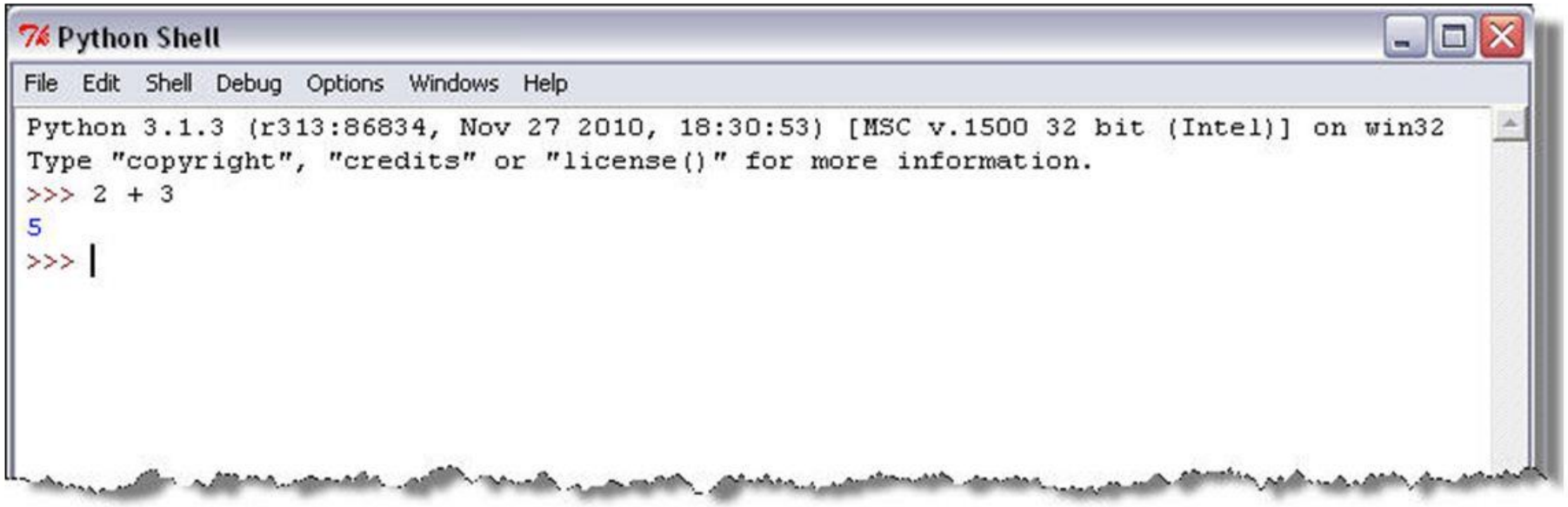
4.1 Python Programming

4.2 How Popular ?

4.3 The IDLE Editor

4.3.1 The Python Shell

- Python can be executed interactively in the **Python shell**.
- The **>>>** symbol is the shell prompt. Here, typing 2 + 3 at prompt outputs the result 5, again displaying the prompt in wait of another instruction



4.3.2 The Python Standard Library

Navigation

4. Introduction To Python Programming

4.1 Python Programming

4.2 How Popular ?

4.3 The IDLE Editor

4.3.1 The Python Shell

4.3.2 The Python Standard Library

- The **Python Standard Library** is a collection of **built in modules**, each providing specific functionality beyond what is included in the “core” part of Python

- **For example**, the math module provides additional mathematical functions. The random module provides the ability to generate random numbers, useful in programming, as we shall see.

- Other Python modules are described in the **Python 3 Programmers’ Reference.**

4.3.2 Importing a Library Module

Navigation

4. Introduction To Python Programming

4.1 Python Programming

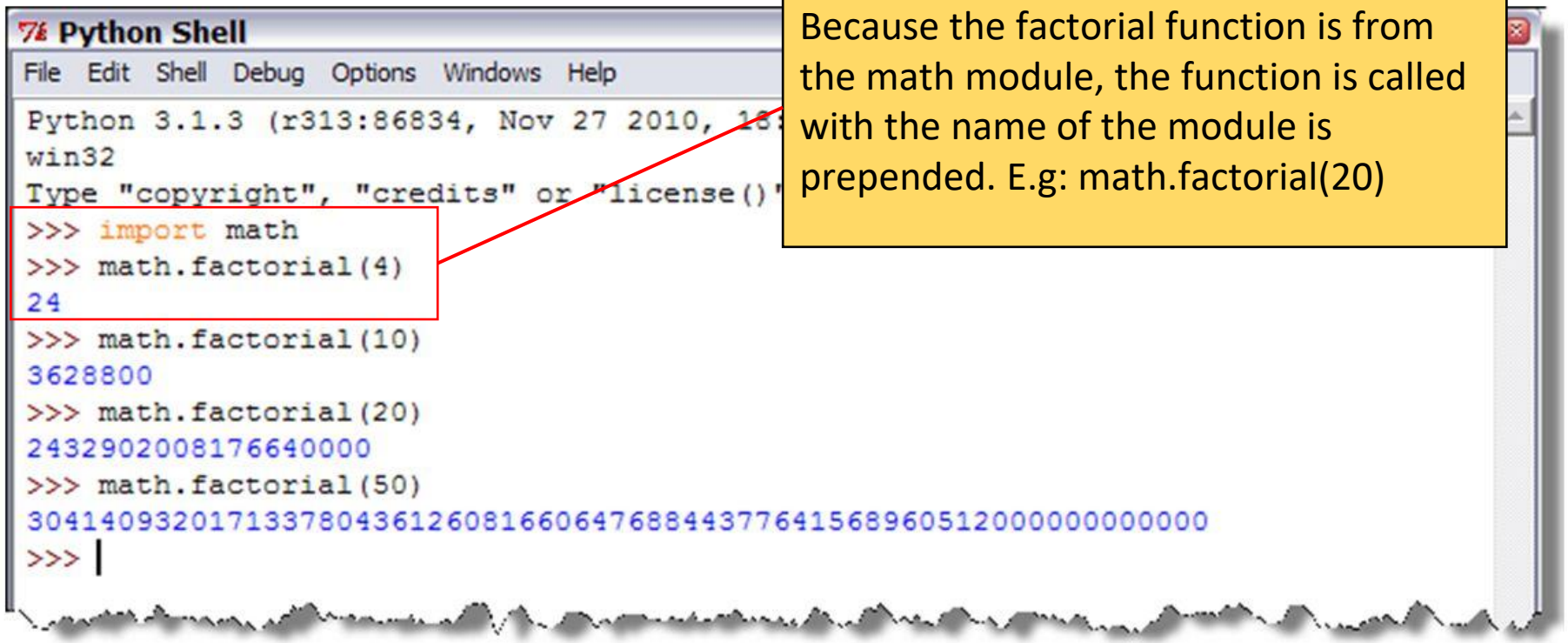
4.2 How Popular ?

4.3 The IDLE Editor

4.3.1 The Python Shell

4.3.2 The Python Standard Library

- In order to utilize the capabilities of modules in a specific program, an import statement is used.



Because the factorial function is from the math module, the function is called with the name of the module is prepended. E.g: math.factorial(20)

5. Some Good Practice

Navigation

5. Some Good Practice

Some of the good practice that you can develop for problem solving skills.

- Daily any 5 simple questions.
- Weekly any 2 challenging questions.
- Monthly any 1 project.

6. Have Some Research!!

Navigation

6. Have Some Research

Before Next Week, Have some research on!!

- What is variables?
- What is arithmetic operators?
- How To Input From User?
- How To Display Output to User?

ANY QUESTIONS?

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing